

ZYNXIS Flutter App

Code Logic Documentation — Navigation & State Flow

This document explains the navigation logic and state handling behind each screen in the app, in support of the task: build a 3-page app (Splash → Login → Dashboard) that passes a **Username** string from the Login page to the Dashboard.

1. Overview

The app is built with Flutter's standard **Navigator** API (no external routing package). Screens are pushed onto a stack using **MaterialPageRoute**. Two navigation methods are used deliberately, depending on whether the previous screen should remain reachable via the back button:

Method	Used for	Effect
Navigator.push()	Dashboard → Home / About / Services / Task Manager	Adds a new screen on top of the stack; back button returns to the previous screen.
Navigator.pushReplacement()	Splash → Login, Login → Dashboard, Dashboard → Login (logout)	Replaces the current screen; the previous screen is removed from the stack (prevents back-navigation to it).

2. Splash Screen — splash.dart

The entry screen. It displays the app branding briefly, then automatically hands off to the Login page. Typical implementation pattern used:

- A `StatefulWidget` starts a `Future.delayed(...)` timer in `initState()`.
- When the timer fires, it calls `Navigator.pushReplacement(context, MaterialPageRoute(builder: (context) => const LoginPage()))`.
- `pushReplacement` is used (not `push`) so the user can never navigate back to the splash screen once past it.

```
@override
void initState() {
  super.initState();
  Future.delayed(const Duration(seconds: 2), () {
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (context) => const LoginPage()),
    );
  });
}
```

3. Login Page — login.dart

A `StatefulWidget` holding one `TextEditingController` (`usernameController`) bound to a `TextField`. Core logic lives in the `login()` method:

1. Reads and trims the text field value into a local `String` `username`.
2. Validation: if `username.isEmpty`, the method returns early and shows a `SnackBar` ('Please enter your username.') — the app never navigates forward with an empty value.
3. On success, it calls `Navigator.pushReplacement(...)`, constructing `DashboardPage` with a required named parameter: `DashboardPage(username: username)`.
4. `login()` is wired to both the Login button's `onPressed` and the `TextField`'s `onSubmitted`, so pressing Enter/Done on the keyboard also triggers navigation.

```
void login() {
  final String username = usernameController.text.trim();

  if (username.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Please enter your username.')),
    );
    return;
  }

  Navigator.pushReplacement(
    context,
    MaterialPageRoute(
      builder: (context) => DashboardPage(username: username),
    ),
  );
}
```

`usernameController` is disposed in `dispose()` to release the text-editing resources when the widget is removed from the tree — standard Flutter controller hygiene.

4. Dashboard — dashboard.dart

A StatelessWidget that **receives** the username passed from Login as a required constructor field:

```
class DashboardPage extends StatelessWidget {  
  final String username;  
  const DashboardPage({super.key, required this.username});  
  ...  
}
```

This is the data hand-off the task requires: the value entered on the Login screen flows through the constructor and is rendered directly in the UI with string interpolation: `Text('Welcome, $username')`.

Other responsibilities of DashboardPage:

- `openPage(context, page)` — a small helper that wraps `Navigator.push(...)` so each `dashboardButton()` only needs to supply the destination widget.
- `dashboardButton({...})` — a reusable builder that renders one tappable card (icon, title, subtitle, arrow) and calls `openPage()` in its `InkWell.onTap`. It is called four times, once per destination (Home, About, Services, Task Manager), each passing a different page: widget.
- The AppBar's logout `IconButton` calls `Navigator.pushReplacement(...)` back to `LoginPage()` — `pushReplacement` again, so the Dashboard (and the now-stale username) is discarded rather than left reachable via back navigation.

5. Dashboard Sub-Pages

Home, About, and Services are StatelessWidgetes that only render static content inside a Scaffold(body: SafeArea(child: SingleChildScrollView(...))) — they hold no navigation logic of their own and return to the Dashboard purely via the automatic back button that `Navigator.push()` provides.

Task Manager (`task_manager.dart`) is the one sub-page with real state. It is a StatefulWidget managing a `List<TaskItem>` plus text controllers and a priority slider value. Key methods:

- `addTask()` — validates the task-name field, then `setState()` to append a new `TaskItem` to the list and clear the inputs.
- `confirmSaveTask()` / `confirmDeleteTask(index)` — show an `AlertDialog` first; only on confirmation do they call `addTask()` / `deleteTask(index)`, so destructive/creating actions are never one accidental tap.
- `deleteTask(index)` — `setState()` to remove the item and show a confirmation `SnackBar`.
- `getPriorityText(priority)` — pure function mapping the 0–100 slider value to Low / Medium / High for display.
- The task list itself is in-memory only (held in State); it resets whenever the page is popped and re-pushed, since no persistence layer is implemented.

6. File-by-File Summary

File	Widget type	Navigation role
splash.dart	StatefulWidget	Entry point → <code>pushReplacement</code> to Login after a delay
login.dart	StatefulWidget	Collects username → <code>pushReplacement</code> to Dashboard(username: ...)
dashboard.dart	StatelessWidget	Receives username; push to 4 sub-pages; <code>pushReplacement</code> to Login on logout
home.dart	StatelessWidget	Leaf screen — back button only

about.dart	StatelessWidget	Leaf screen — back button only
services.dart	StatelessWidget	Leaf screen — back button only
task_manager.dart	StatefulWidget	Leaf screen — local CRUD state, back button only

See [navigation_flow_diagram.pdf](#) for the accompanying visual diagram of this flow.